

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	Ch.saisri	Reg. No.:	192472367
Section / Batch:	Cse-ai	Date:	

Code of Conduct

*I **ch.saisri(192472367)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging, HMM	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q1

Annexure A – SCENARIO BASED

A company is developing an NLP-based grammar checker for educational applications. The system is trained using a manually POS-tagged corpus. Your task is to recreate the Hidden Markov Model (HMM) **without using any built-in NLP or HMM libraries**.

From the given corpus,

- determine the **Emission Probability**
- determine the **Transition Probability**
- implement the **Viterbi Algorithm**
- predict the POS tags for the new input sentence.

Use only Python dictionaries, loops, and basic programming constructs.

Training Corpus

Sentence	POS Tags
The/DT boy/NN eats/VBZ rice/NN	DT NN VBZ NN
The/DT girl/NN drinks/VBZ milk/NN	DT NN VBZ NN
A/DT cat/NN drinks/VBZ milk/NN	DT NN VBZ NN
The/DT dog/NN chases/VBZ cat/NN	DT NN VBZ NN
A/DT teacher/NN teaches/VBZ students/NNS	DT NN VBZ NNS
Students/NNS study/VBP English/NN	NNS VBP NN
Birds/NNS fly/VBP high/RB	NNS VBP RB
Children/NNS play/VBP games/NNS	NNS VBP NNS

New Input Sentence

The cat drinks milk

Tasks

1. Separate each sentence into words and POS tags.
2. Compute the **Emission Probability** of every word.
3. Compute the **Transition Probability** between POS tags.
4. Recreate the Hidden Markov Model using Python dictionaries.
5. Implement the **Viterbi Algorithm** without using any built-in HMM/NLP libraries.
6. Predict the POS tag sequence for “The cat drinks milk”

Code :

Hidden Markov Model (HMM) using only Python Dictionaries

```
training_data = [  
    ("The boy eats rice", ["DT", "NN", "VBZ", "NN"]),  
    ("The girl drinks milk", ["DT", "NN", "VBZ", "NN"]),  
    ("A cat drinks milk", ["DT", "NN", "VBZ", "NN"]),  
    ("The dog chases cat", ["DT", "NN", "VBZ", "NN"]),  
    ("A teacher teaches students", ["DT", "NN", "VBZ", "NNS"]),  
    ("Students study English", ["NNS", "VBP", "NN"]),  
    ("Birds fly high", ["NNS", "VBP", "RB"]),  
    ("Children play games", ["NNS", "VBP", "NNS"])  
]
```

Dictionaries

emission_count = {}

transition_count = {}

tag_count = {}

start_count = {}

Step 1: Separate words and POS tags

for sentence, tags in training_data:

```

words = sentence.split()

start_tag = tags[0]
start_count[start_tag] = start_count.get(start_tag, 0) + 1

for word, tag in zip(words, tags):

    if tag not in emission_count:
        emission_count[tag] = {}

    emission_count[tag][word] = emission_count[tag].get(word, 0) + 1

    tag_count[tag] = tag_count.get(tag, 0) + 1

for i in range(len(tags)-1):

    current = tags[i]
    nxt = tags[i+1]

    if current not in transition_count:
        transition_count[current] = {}

    transition_count[current][nxt] = transition_count[current].get(nxt, 0) + 1

# -----
# Step 2: Emission Probability
# -----

print("\nEMISSION PROBABILITIES\n")

emission_prob = {}

for tag in emission_count:

    emission_prob[tag] = {}

    for word in emission_count[tag]:

        probability = emission_count[tag][word] / tag_count[tag]

        emission_prob[tag][word] = probability

        print(f"P({word}|{tag}) = {probability:.3f}")

# -----
# Step 3: Transition Probability
# -----

print("\nTRANSITION PROBABILITIES\n")

transition_prob = {}

for tag in transition_count:

    transition_prob[tag] = {}

    total = sum(transition_count[tag].values())

    for nxt in transition_count[tag]:

        probability = transition_count[tag][nxt] / total

        transition_prob[tag][nxt] = probability

```

```

    print(f"P({nxt})[{tag}] = {probability:.3f}")

# -----
# Step 4: Hidden Markov Model
# -----

print("\nHIDDEN MARKOV MODEL\n")

print("States (POS Tags):")

for tag in tag_count:
    print(tag)

print("\nEmission Dictionary")

print(emission_prob)

print("\nTransition Dictionary")

print(transition_prob)

# -----
# Step 5: Viterbi Algorithm
# -----

sentence = "The cat drinks milk".split()

states = list(tag_count.keys())

V = []

path = {}

# Initialization

V.append({})

for state in states:

    start_probability = start_count.get(state,0)/len(training_data)

    emission = emission_prob.get(state,{}).get(sentence[0],0.0001)

    V[0][state] = start_probability * emission

    path[state] = [state]

# Recursion

for t in range(1,len(sentence)):

    V.append({})

    new_path = {}

    for current_state in states:

        max_prob = -1

        best_state = None

        emission = emission_prob.get(current_state,{}).get(sentence[t],0.0001)

        for previous_state in states:

```

```

transition = transition_prob.get(previous_state,{}).get(current_state,0.0001)

probability = V[t-1][previous_state] * transition * emission

if probability > max_prob:

    max_prob = probability

    best_state = previous_state

V[t][current_state] = max_prob

new_path[current_state] = path[best_state] + [current_state]

path = new_path

# Termination

maximum = -1

best_final = None

for state in states:

    if V[-1][state] > maximum:

        maximum = V[-1][state]

        best_final = state

print("\nPredicted POS Tags\n")

print(sentence)

print(path[best_final])

```

```

EMISSION PROBABILITIES

P(The|DT) - 0.600
P(A|DT) - 0.400
P(boy|NN) - 0.100
P(rice|NN) - 0.100
P(girl|NN) - 0.100
P(milk|NN) - 0.200
P(cat|NN) - 0.200
P(dog|NN) - 0.100
P(teacher|NN) - 0.100
P(English|NN) - 0.100
P(eats|VBZ) - 0.200
P(drinks|VBZ) - 0.400
P(chases|VBZ) - 0.200
P(teaches|VBZ) - 0.200
P(students|NNS) - 0.200
P(Students|NNS) - 0.200
P(Birds|NNS) - 0.200
P(Children|NNS) - 0.200
P(games|NNS) - 0.200
P(study|VBP) - 0.333
P(fly|VBP) - 0.333
P(play|VBP) - 0.333
P(high|RB) - 1.000

TRANSITION PROBABILITIES

P(NN|DT) - 1.000
P(VBZ|NN) - 1.000
P(NN|VBZ) - 0.800
P(NNS|VBZ) - 0.200
P(VBP|NNS) - 1.000
P(NN|VBP) - 0.333
P(RB|VBP) - 0.333
P(NNS|VBP) - 0.333

HIDDEN MARKOV MODEL

States (POS Tags):
DT
NN
VBZ
NNS
VBP
RB

Emission Dictionary
{'DT': {'The': 0.6, 'A': 0.4}, 'NN': {'boy': 0.1, 'rice': 0.1, 'girl': 0.1, 'milk': 0.2, 'cat': 0.2, 'dog': 0.1, 'teacher': 0.1, 'English': 0.1}, 'VBZ': {'eats': 0.2, 'drinks': 0.4, 'chases': 0.2, 'teaches': 0.2}, 'NNS': {'students': 0.2, 'Students': 0.2, 'Birds': 0.2, 'Children': 0.2, 'games': 0.2}, 'VBP': {'study': 0.3333333333333333, 'fly': 0.3333333333333333, 'play': 0.3333333333333333}, 'RB': {'high': 1.0}}

Transition Dictionary
{'DT': {'NN': 1.0}, 'NN': {'VBZ': 1.0}, 'VBZ': {'NN': 0.8, 'NNS': 0.2}, 'NNS': {'VBP': 1.0}, 'VBP': {'NN': 0.3333333333333333, 'RB': 0.3333333333333333, 'NNS': 0.3333333333333333}}

Predicted POS Tags
['The', 'cat', 'drinks', 'milk']
['DT', 'NN', 'VBZ', 'NN']

--- Code Execution Successful ---

```

SSSXS

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Understanding of Scenario	Identification of Key Issues	Application of Concepts	Decision Making	Clarity of Response	Sub. Total	Total %
1	4	4	4	3	3	20	100
2	4	3	4	3	4	20	
3	3	4	3	4	4	25	
4	3	3	4	4	4	20	
5	4	4	4	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____